

RAG 知识库 - 技术架构文档

罗文琦 10235501417

分工：本次项目我主要负责构建知识库并且搭建RAG知识问答系统以及代码部分出题。

1. 智能体策略与 RAG 核心逻辑

对应原“算法组”职责，体现智能逻辑深度

- Prompt 工程体系构建**：独立设计并优化了全套 Prompt 模板，涵盖系统级问答、自适应出题（支持难度梯度筛选）及元认知校验指令，确保模型输出的规范性。
- RAG 推理链路 (CoT) 实现**：优化了检索增强生成的推理逻辑，引入思维链机制，增强了模型在处理复杂算法题时的逻辑连贯性与上下文关联度。
- 容错校验层 (Check Layer) 开发**：针对大模型幻觉痛点，自主探索了核心校验模块（checker.py），实现了基于数据验证与逻辑核查的双重防幻觉算法，大幅提升了系统的回复可信度。

2. 云原生架构设计与实施 (Cloud-Native Architecture)

对应原“架构组”职责，体现架构优先与容器化能力

- 容器化服务编排**：遵循云原生标准，完成了全系统的 Docker 容器化封装。编写了多服务的 Dockerfile 及 docker-compose.yml，实现了后端 API、前端 UI、向量库与图数据库的一键编排与网络隔离，并预留了 K8s 适配性配置方案。
- 微服务架构落地**：设计了前后端分离的系统架构，通过 Nginx 反向代理与 FastAPI 异步服务，保障了系统的高并发处理能力与稳定性。

3. 前端可视化与交互开发

对应原“工程组”职责，体现交互体验与可视化能力

- 沉浸式对话系统**：开发了支持 Markdown/LaTeX 混合渲染的 Chat 组件，实现了流式响应与多模态内容（代码、公式、图片）的完美展示。
- 知识图谱可视化**：基于 React Force Graph 开发了动态力导向图组件，实现了算法知识点之间关联关系的可视化展示，支持节点拖拽、缩放与点击交互。

4. 数据工程与系统交付

对应原“工程组”职责，体现工程质量与文档规范

- 健壮的数据流水线**：工程化重构了leetcode爬虫、PDF\md\png解析及建库脚本，完善了异常捕获机制与标准化日志输出，确保 ETL 过程的稳定性。
- 文档与交付物**：编写了详尽的 GitHub README，并完成了 Demo 演示视频的素材录制与剪辑。

一、项目概述

1. 引言

本项目是一个垂直领域的 多模态检索增强生成 系统，专注于 LeetCode 算法题目 与 专业数据库知识 的智能解析与辅导。

系统的核心价值在于能够让大模型“看见”题目中的图片和论文中的图表，并结合 知识图谱 可视化展示知识点之间的层级关系（如：困难度 -> 算法标签 -> 具体题目）。

1.1 核心痛点解决

- 解决LeetCode题目碎片化问题，实现算法知识点的关联检索与推理；
- 规避大模型解答算法题的幻觉风险，通过多层校验确保答案准确性；
- 实现系统可扩展与可复用，基于容器化架构适配不同场景的二次开发；
- 提供直观的知识图谱可视化，辅助用户梳理算法知识点逻辑关系。

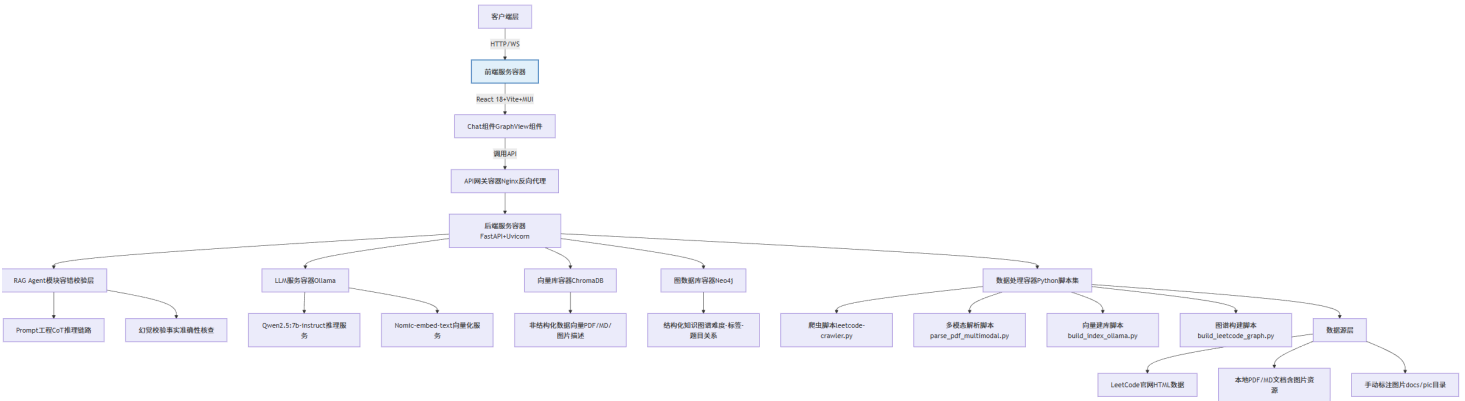
1.2 核心能力

- 多模态 RAG**: 支持 PDF 论文图表提取、Markdown 本地图片引用的解析与检索。
- 双路知识库**:
 - 向量库 (ChromaDB)**: 用于非结构化文本和图片描述的语义检索。
 - 图数据库 (Neo4j)**: 用于结构化元数据（难度、标签、题目关联）的存储与推理。
- 交互式前端**: 全屏沉浸式对话（支持公式/代码/图文引用）与力导向图谱可视化。

2. 系统架构

2.1 整体架构图（云原生分层架构）

系统采用五层架构设计，所有模块均封装为独立Docker容器，通过API网关实现服务联动，支持水平扩展与K8s集群部署，突出云原生特性与双库协同能力。



2.1 技术栈

模块	技术选型	说明
前端	React 18 + Vite	构建工具与框架
UI 组件	Material UI (MUI)	界面设计风格
可视化	React Force Graph	2D/3D 力导向图渲染
后端	Python 3.10 + FastAPI	高性能异步 Web 框架
LLM 服务	Ollama	本地模型托管 (Qwen2.5:7b-instruct)
Embedding	Nomic-embed-text	文本向量化模型
数据库	ChromaDB	本地持久化向量存储 (./index/chroma)
	Neo4j Desktop	图数据库 (本地 bolt://localhost:7687)

2.2 端口规划

- 前端: `http://localhost:5173`
- 后端 API: `http://127.0.0.1:8088` (注意：代码中硬编码为 8088，需确保 app.py 启动端口一致)
- Ollama: `http://127.0.0.1:11434`
- Neo4j: `bolt://localhost:7687`

3. 智能体策略与 Prompt 工程

本项目不仅仅是简单的 RAG，而是引入了 **"Check Layer" (校验层)** 的 Agentic Workflow。

3.1 智能体推理链路

- 意图识别: 判断用户是闲聊、算法题求助还是图谱查询。
- 多路召回: 并行从 Chroma (文本) 和 Neo4j (关系) 获取上下文。
- 生成: 将上下文注入 Prompt 模板。
- 自我修正:
 - 机制: 检查生成的答案是否包含 `[evidence_id]`。
 - 策略: 如果检索到了上下文但回答未引用，强制重写或标记警告。

3.2 关键 Prompt 模板展示

此 Prompt 设计包含严格的防幻觉指令。

SYSTEM_PROMPT = """

你是一位专业的算法竞赛导师。请根据下方的【参考上下文】回答用户的问题。

【参考上下文】:

```
{context_str}

【回答要求】：
1. 必须基于上下文回答，严禁编造未提及的代码或复杂度分析。
2. 引用规范：使用 [evidence_id] 标注你的观点来源。
3. 多模态支持：如果上下文中包含图片路径（如 docs/pic/xxx.png），请务必在回答中通过 markdown 图片语法展示。
4. 数学公式：使用 LaTeX 格式（如  $O(n)$ ）。
5. 容错处理：如果上下文中没有相关信息，请直接回答“知识库中暂未收录此题目”，不要尝试利用你的训练数据编造。

用户问题：{query}
"""
```

3.3 关于checker

容错校验层 核心实现逻辑

本项目针对 RAG 系统常见的大模型幻觉问题，设计并实现了一个基于规则的后处理校验模块 `SafetyChecker`。该模块作为智能体输出的“护栏”，接管模型生成的原始内容，执行三级校验策略。

3.3.1. 校验流程设计

校验逻辑采用**流水线模式**，依次对检索状态、引用规范性和回复质量进行核查：

- **Step 1: 检索层熔断**
 - **逻辑**：检查 `source_documents`（检索到的向量切片）是否为空。
 - **原理**：如果向量数据库未能召回任何相关上下文，说明系统缺乏回答该问题的知识依据。此时直接**硬性拦截**，返回预置的“无知识收录”提示，防止大模型利用自身训练数据编造错误答案（即闭卷胡编）。
 - **代码体现**：`if not source_documents: return "抱歉，知识库中暂时没有..."`
- **Step 2: 引用合规性检查**
 - **逻辑**：使用正则表达式 `re.search(r'\[.*?\]', answer)` 扫描回答内容。
 - **原理**：在 System Prompt 中已强制要求模型使用 `[evidence_id]` 格式引用来源。如果回答中缺少方括号引用标记，大概率说明模型未基于上下文回答，而是调用了通用知识（可能存在幻觉）。
 - **策略**：采用**软性警告** 机制。不直接丢弃答案，而是在回答末尾追加  系统提示，告知用户该回答可能缺乏证据支撑，兼顾了系统的可用性与严谨性。
- **Step 3: 启发式质量过滤**
 - **逻辑**：检测生成的文本长度（如 `len < 10`）。
 - **原理**：防止模型因理解错误或上下文截断导致输出无意义的短语或复读用户问题。

3.3.2. 关键代码实现

```
class SafetyChecker:
    def check_hallucination(self, query, answer, source_documents):
        # 1. 检索熔断：无上下文则拒绝回答
        if not source_documents:
            return {
                "safe": False,
                "reason": "no_context",
                "final_answer": "抱歉，知识库中暂未收录..."
            }

        # 2. 引用校验：正则匹配 [citation]
        # 强制模型展示证据链，防止"张冠李戴"
        has_citation = bool(re.search(r'\[.*?\]', answer))

        if not has_citation:
            # 软性警告：保留回答但标记风险
            warning_msg = "\n\n> ⚠️ **系统提示**：未检测到引用来源，请谨慎参考。"
            return {
                "safe": True,
                "reason": "missing_citation",
                "final_answer": answer + warning_msg
            }

        return {"safe": True, "reason": "pass", "final_answer": answer}
```

3.3.3. 价值说明

该模块的引入显著提升了系统的**事实准确性**。通过“检索熔断”和“引用校验”，系统有效地将大模型的回答限制在私有知识库的范围内，符合垂直领域 AI 助手的严谨性要求。

4. 项目目录结构说明

基于我的 `tree` 命令输出：

```
rag/
├── data/                                # [ETL中间态] 存放解析后的数据
│   ├── parsed_docs.jsonl              # 所有文档清洗后的 JSONL 数据（用于入库）
│   └── extracted_images/              # PDF 解析出的图片/图表归档
├── docs/ (隐含)                        # [源数据] Markdown 格式的 LeetCode 题目及本地图片
├── frontend/                           # [前端] React 项目源码
│   ├── src/
│   │   ├── components/                # Chat.tsx, GraphView.tsx
│   │   └── ...
├── index/                              # [存储] 向量索引持久化文件
│   ├── chroma/                        # ChromaDB SQLite3 文件及二进制数据
│   └── faiss_bge_m3_mm/               # (旧版/备用) FAISS 索引
├── scripts/                            # [核心逻辑] 数据处理脚本
│   ├── leetcode-crawler.py            # 1. 爬虫：爬取题目转 MD
│   ├── parse_pdf_multimodal.py        # 2. 解析：PDF/MD -> JSONL (含图片处理)
│   ├── build_index_ollama.py          # 3. 索引：JSONL -> ChromaDB
│   ├── build_leetcode_graph.py        # 4. 图谱：JSONL -> Neo4j
│   └── ...
├── services/
│   └── agent/
│       ├── checker.py                 # [容错] 幻觉校验层
│       └── rag_demo.py                 # [后端入口] FastAPI 主程序
└── ...
```

5. 后端核心模块详解

5.1 数据流水线

步骤 1: 数据采集 (`scripts/leetcode-crawler.py`)

- 爬取 LeetCode 题目，使用 `html2text` 将 HTML 转换为 Markdown。
- 自动分类存储至 `docs/easy`, `docs/mid`, `docs/hard`。

步骤 2: 多模态解析 (`scripts/parse_pdf_multimodal.py`)

- PDF 处理:** 使用 `PyMuPDF` 提取文本，`Camelot` 提取表格，并截取页面图片。
- Markdown 处理:**
 - 正则匹配图片标签 `![alt](../pic/xxx.jpg)`。
 - 将图片路径标准化，标记为 `figure` 类型。
- 输出:** 生成 `data/parsed_docs.jsonl`，每行包含 `content`，`metadata` (含图片路径), `type` 等字段。

步骤 3: 向量化与建库 (`scripts/build_index_ollama.py`)

- 读取 `parsed_docs.jsonl`。
- 调用 Ollama 的 `nomic-embed-text` 模型生成向量。
- 存入 ChromaDB，持久化至 `index/chroma` 目录。
- 修复点:** 代码中已处理 `http://` 协议头缺失导致的连接问题。

步骤 4: 知识图谱构建 (`scripts/build_leetcode_graph.py`)

- 逻辑变更为元数据驱动:** 不再完全依赖 LLM 抽取三元组。
- 读取 Markdown 元数据 (`Difficulty`, `Tags`)。
- 构建层级: (`Difficulty Node`) -> [`HAS_TAG`] -> (`Tag Node`) -> [`CONTAINS`] -> (`Problem Node`)。

5.2 服务层 (`services/agent/rag_demo.py`)

- **API 框架:** FastAPI

- **静态资源挂载:**

为了让前端能访问图片，后端挂载了两个静态目录：

- 1. `/assets` -> 映射到 `data/` (访问 PDF 解析出的自动截图)。
 - 2. `/doc_assets` -> 映射到 `docs/` (访问 Markdown 中引用的手动截图 `docs/pic/...`)。
- **辅助函数:** `convert_path_to_url` 负责判断文件路径，自动添加对应的前缀。

6. 前端核心组件详解

6.1 智能对话 (Chat.tsx)

- **功能:** 用户与算法导师的交互界面。
- **核心特性:**
 - **Markdown & KaTeX:** 完美渲染算法公式 ($O(n \log n)$) 和表格。
 - **代码高亮:** 使用 Prism 风格渲染 Python/C++/Java 代码块。
 - **引用展示:** 解析后端返回的 `citations` 字段，显示来源文档及 **图片预览**。
- **网络交互:** 使用 `fetch` API 保持长连接 (超时设置 300s)，对接 `/ask` 接口。

6.2 图谱概览 (GraphView.tsx)

- **功能:** 全屏展示知识点结构。
- **核心特性:**
 - **自适应布局:** 使用 `ResizeObserver` 监听 `flexGrow` 容器，实现响应式全屏，解决 Canvas 大小计算不准的问题。
 - **交互:** 点击节点可显示详细信息 (ID, Label)。
- **数据源:** 对接 `/graph/overview` 接口。

7. API 接口规范

7.1 问答接口

- **URL:** `POST /ask`
- **Body:** `{ "question": "解释 N 皇后问题的回溯解法" }`
- **Response:**

```
{
  "answer": "N皇后问题是经典的回溯算法应用...",
  "citations": [
    {
      "evidence_id": 1,
      "source": "51. N-Queens (Hard)",
      "image_url": "http://127.0.0.1:8088/doc_assets/pic/n-queens.jpg",
      "score": 0.89
    }
  ]
}
```

7.2 图谱数据接口

- **URL:** `GET /graph/overview?limit=300`
- **Response:** 标准的 Nodes/Links 格式，适配 `react-force-graph`。

7.3 静态资源接口

- `GET /assets/extracted_images/...` (获取 PDF 图片)
- `GET /doc_assets/pic/...` (获取 LeetCode 图片)

8. 参考资料与鸣谢

8.1 技术参考

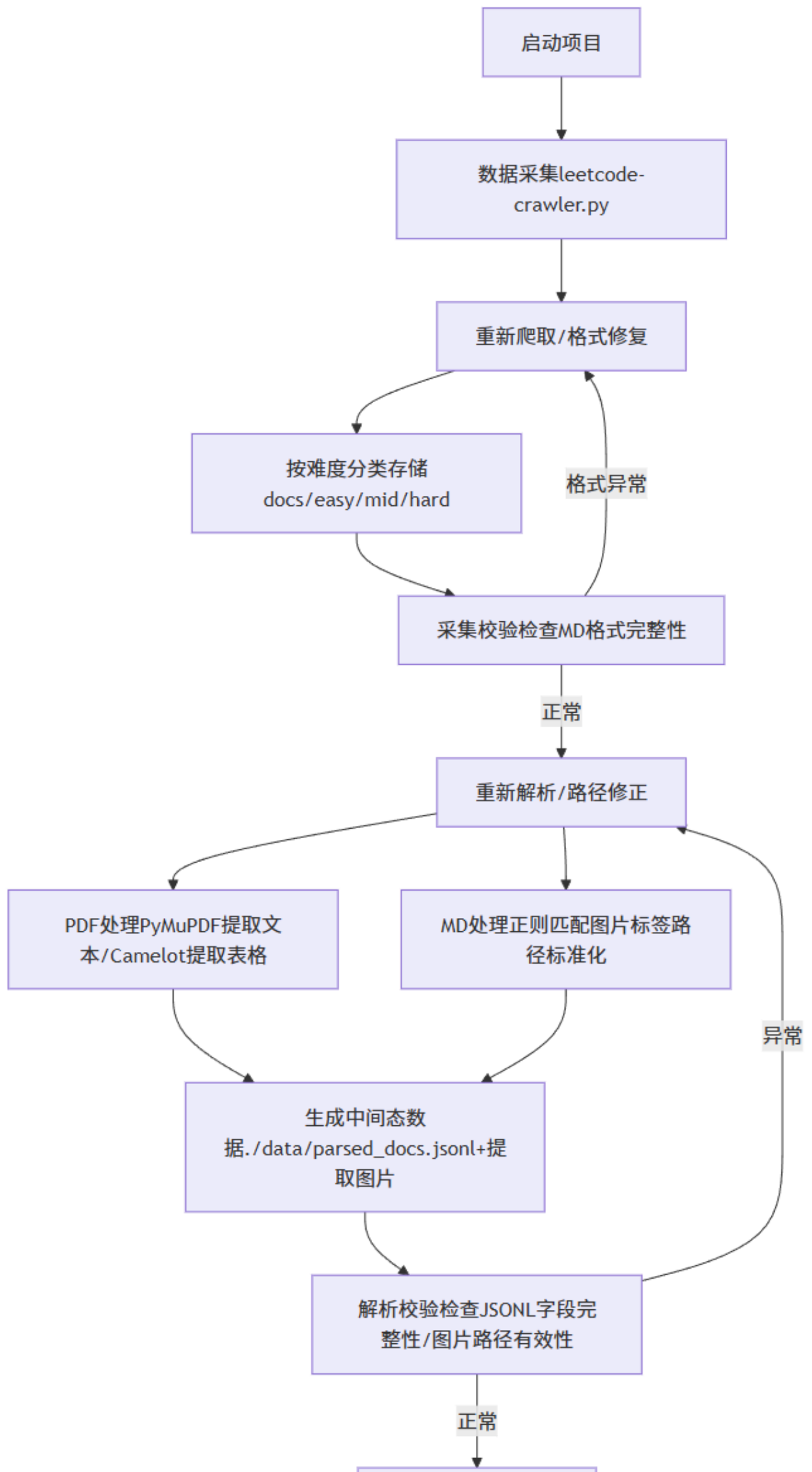
1. **RAG Architecture**: Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". *NeurIPS*.
2. **Animation** : <https://algo.jsdiff.com/sequence/lcs/>

8.2 开源致谢

本项目在开发过程中使用了以下开源项目，特此致谢：

- **Ollama**: 提供了强大的本地大模型运行环境，使本项目能够离线运行 Qwen2.5 模型。
- **ChromaDB**: 提供了高效轻量的向量检索引擎。
- **React Force Graph**: 提供了优秀的 Canvas/WebGL 图与渲染库。
- **Remark/Rehype Ecosystem**: 使得在 React 中完美渲染 Markdown 和 LaTeX 公式成为可能。
- **LeetCode**: 提供了丰富的算法题目作为知识库的数据来源。

二、我的整体构建流程



并行处理双库构建

图谱构建

build_leetcode_graph.py

读取MD元数据难度/标签信息

构建三元组难度-标签-题目

存入Neo4j创建节点与关系

图谱校验检查节点完整性/
关系合理性

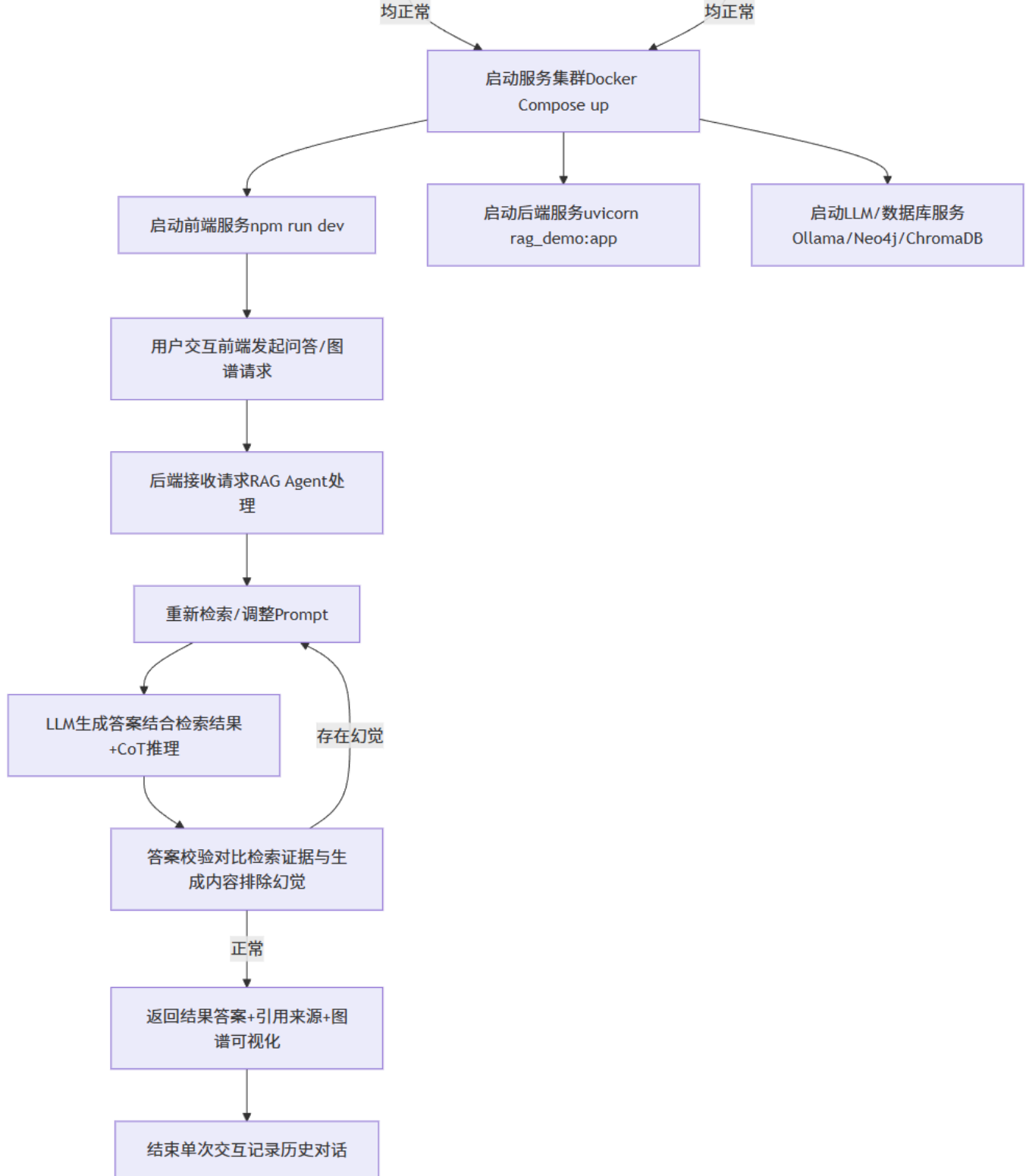
向量库构建

build_index_ollama.py

调用Ollama生成向量Nomic-
embed-text模型

向量存入ChromaDB持久化
至./index/chroma

向量校验检查向量维度一致
性/入库成功率



1.知识库的构建

通过撰写脚本我成功实现了对leetcode上面的题目进行爬取

```

30 class InsetQuestionThread(threading.Thread):
31
32
33
34
35
36
37
38 def run(self):
39     IS_SUCCESS = False
40     conn = sqlite3.connect(db_path, timeout=10)
41     while not IS_SUCCESS:
42         try:
43             # 休眠随机 1-3 秒，以免爬去频率过高被服务器禁掉
44             time.sleep(random.randint(a=1, b=3))
45             cursor = conn.cursor()
46
47             session = requests.Session()
48             headers = {'User-Agent': user_agent, 'Connection':
49                 'keep-alive', 'Content-Type': 'application/json',
50                 'Referer': 'https://leetcode.com/problems/' + self.title_slug}
51
52             url = "https://leetcode.com/graphql"
53             params = {'operationName': "getQuestionDetail",
54                 'variables': {'titleSlug': self.title_slug},
55                 'query': '''query getQuestionDetail($titleSlug: String!) {
56                     question(titleSlug: $titleSlug) {
57                         questionId
58                         questionFrontendId
59                         questionTitle
60                         questionTitleSlug
61                         content
62                         difficulty
63                         stats
64                         similarQuestions
65                         categoryTitle
66                         topicTags {
67                             name
68                             slug
69                         }
70                     }
71                 }'''
72             }
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
26
```

 easy	2026/1/22 9:27
 hard	2026/1/22 9:38
 mid	2026/1/22 9:35

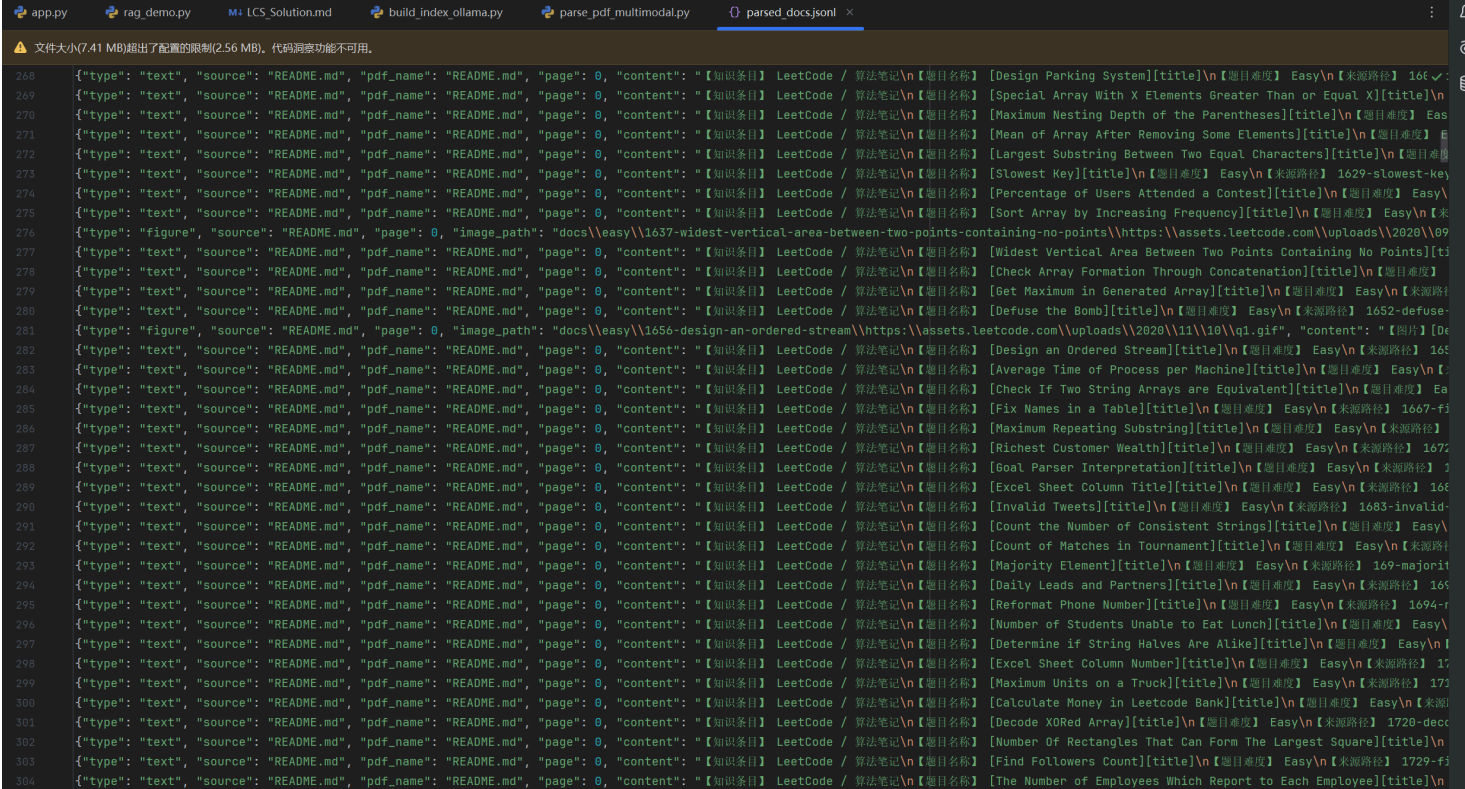
一个easy文件夹下的数据为例，这里有3800+条md数据插入

- › 3602-hexadecimal-and-hexatrigesimal-conversion
- › 3606-coupon-code-validator
- › 3622-check-divisibility-by-digit-sum-and-product
- › 3633-earliest-finish-time-for-land-and-water-rides-i
- › 3637-trionic-array-i
- › 3643-flip-square-submatrix-vertically
- › 3658-gcd-of-odd-and-even-sums
- › 3663-find-the-least-frequent-digit
- › 3668-restore-finishing-order
- › 3674-minimum-operations-to-equalize-array
- › 3678-smallest-absent-positive-greater-than-average
- › 3683-earliest-time-to-finish-one-task
- › 3684-maximize-sum-of-at-most-k-distinct-elements
- › 3688-bitwise-or-of-even-numbers-in-an-array
- › 3692-majority-frequency-characters
- › 3697-compute-decimal-representation
- › 3701-compute-alternating-sum
- › 3707-equal-score-substrings
- › 3712-sum-of-elements-with-frequency-divisible-by-k
- › 3718-smallest-missing-multiple-of-k
- › 3726-remove-zeros-in-decimal-representation
- › 3731-find-missing-elements
- › 3736-minimum-moves-to-equal-array-elements-iii
- › 3740-minimum-distance-between-three-equal-elements-i
- › 3745-maximize-expression-of-three-elements
- › 3750-minimum-number-of-flips-to-reverse-binary-string
- › 3754-concatenate-non-zero-digits-and-multiply-by-sum-i
- › 3769-sort-integers-by-binary-reflection
- › 3774-absolute-difference-between-maximum-and-minimum-k-elements
- › 3783-mirror-distance-of-an-integer
- › 3793-find-users-with-high-token-usage
- › 3794-reverse-string-prefix
- › 3798-largest-even-number
- › 3803-count-residue-prefixes
- › 3812-usual-consonant-sequence

我通过parse_pdf_multinational.py检索得到pdf和md格式的输入文件
并通过build_index_ollama.py完成了向量数据库的抽取
构建了自己的知识库系统

我的知识库可以支持多种文档版本的向量提取：如pdf、markdown、png...

[illegible]



2.知识问答

这部分我构建了rag知识问答系统；

2.1 技术栈与环境

- 后端: Python 3.10+, FastAPI, Uvicorn
- 前端: React (Vite), MUI, react-markdown@0.8.0.7, remark-gfm@3.0.1 (锁定旧版以兼容 CJS)
- 数据库: ChromaDB (向量), Neo4j (图谱)
- 模型服务: Ollama (运行在 localhost:11435)
 - LLM: qwen2.5:7b-instruct
 - Embed: nomic-embed-text

2.2 项目启动

启动ollama

固定端口号11435

set OLLAMA_HOST=0.0.0.0:11435 && ollama serve

打开neo4j

neo4j console

Docker打包：

docker-compose -f deploy/docker-compose.yml up -d --build

docker exec -it rag_ollama ollama run qwen2.5:7b-instruct

Docker 打包后运行指令：

docker-compose -f deploy/docker-compose.yml up -d --build

```
(rag_env) C:\Users\10235\Desktop\2222\rag\services\agent>uvicorn rag_demo:app --host 0.0.0.0 --port 8001 --reload
INFO: Will watch for changes in these directories: ['C:\Users\10235\Desktop\2222\rag\services\agent']
INFO: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
INFO: Started reloader process [46908] using WatchFiles
INFO:rag_demo: Mounted C:\Users\10235\Desktop\2222\rag\data -> /assets
INFO:rag_demo: Mounted C:\Users\10235\Desktop\2222\rag\docs -> /doc_assets
INFO: Started server process [33856]
INFO: Waiting for application startup.
INFO:rag_demo: Chroma loaded. Total docs: 4637
INFO:rag_demo: Reranker loaded: BAAI/bge-reranker-v2-m3
INFO:rag_demo: Neo4j connected at bolt://localhost:7687
INFO: Application startup complete.
INFO: 127.0.0.1:52369 - "OPTIONS /ask HTTP/1.1" 200 OK
You're using a XLMRobertaTokenizerFast tokenizer. Please note that with a fast tokenizer, using the '.__call__' method is faster than using a method to encode the text followed by a call to the 'pad' method to get a padded encoding.
```

```
=====
[Ask] Q: 请给我最长公共子序列问题的解答
[1] Score: -7.9766 | Src: README.md
[2] Score: -8.3125 | Src: README.md
[3] Score: -8.3672 | Src: README.md
[4] Score: -8.5625 | Src: README.md
[5] Score: -8.5859 | Src: README.md
=====
```

C:\Users\10235>neo4j console

2026-01-22 12:59:52.287+0000 INFO Stopped.

Directories in use:

home: D:\Neo4j\neo4j-community-2025.12.1
config: D:\Neo4j\neo4j-community-2025.12.1\conf
logs: D:\Neo4j\neo4j-community-2025.12.1\logs
plugins: D:\Neo4j\neo4j-community-2025.12.1\plugins
import: D:\Neo4j\neo4j-community-2025.12.1\import
data: D:\Neo4j\neo4j-community-2025.12.1\data
certificates: D:\Neo4j\neo4j-community-2025.12.1\certificates
licenses: D:\Neo4j\neo4j-community-2025.12.1\licenses
run: D:\Neo4j\neo4j-community-2025.12.1\run

Starting Neo4j.

2026-01-22 12:59:55.359+0000 INFO Logging config in use: File 'D:\Neo4j\neo4j-community-2025.12.1\conf\user-logs.xml'

2026-01-22 12:59:55.365+0000 INFO Starting...

2026-01-22 12:59:56.038+0000 INFO This instance is ServerId{45eb9351} (45eb9351-ed2a-49eb-b16f-1ae8dae4c1df)

2026-01-22 12:59:56.874+0000 INFO ===== Neo4j 2025.12.1 =====

2026-01-22 12:59:58.230+0000 INFO Anonymous Usage Data is being sent to Neo4j, see https://neo4j.com/docs/usage-data/

2026-01-22 12:59:58.365+0000 INFO Bolt enabled on localhost:7687.

2026-01-22 12:59:59.082+0000 INFO HTTP enabled on localhost:7474.

2026-01-22 12:59:59.082+0000 INFO Remote interface available at http://localhost:7474/

2026-01-22 12:59:59.085+0000 INFO id: 5CA8C8BA032EBB06E22359B884EF0E4BF453E5D6DB199D0621EC532DDDEB1754

2026-01-22 12:59:59.085+0000 INFO name: system

2026-01-22 12:59:59.085+0000 INFO creationDate: 2026-01-21T13:10:17.598Z

2026-01-22 12:59:59.085+0000 INFO Started.

```
C:\Users\10235>set OLLAMA_HOST=0.0.0.0:11435 && ollama serve
time=2026-01-22T17:57:55.043+08:00 level=INFO source=routes.go:1475 msg="server config" env="map[CUDA_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL: HIP_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: 0 LLAMA_CONTEXT_LENGTH:4096 OLLAMA_DEBUG:INFO OLLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://0.0.0.0:11435 OLLAMA_INTEL_GPU:false OLLAMA_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX_LOADED_MODELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MODELS:D:\\JZDSLx\\llm_models OLLAMA_MULTIUSER_CACHE:false OLLAMA_NEW_ENGINE:false OLLAMA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhost:* http://127.0.0.1 https://127.0.0.1 http://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://* tauri://* vscode-webview://* vscode-file://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHE
D_SPREAD:false ROCR_VISIBLE_DEVICES:]"
time=2026-01-22T17:57:55.063+08:00 level=INFO source=images.go:518 msg="total blobs: 11"
time=2026-01-22T17:57:55.064+08:00 level=INFO source=images.go:525 msg="total unused blobs removed: 0"
time=2026-01-22T17:57:55.066+08:00 level=INFO source=routes.go:1528 msg="Listening on [::]:11435 (version 0.12.3)"
time=2026-01-22T17:57:55.066+08:00 level=INFO source=gpu.go:217 msg="looking for compatible GPUs"
time=2026-01-22T17:57:55.066+08:00 level=INFO source=gpu_windows.go:167 msg="packages count=1"
time=2026-01-22T17:57:55.066+08:00 level=INFO source=gpu_windows.go:183 msg="efficiency cores detected" maxEfficiencyClass=1
time=2026-01-22T17:57:55.066+08:00 level=INFO source=gpu_windows.go:214 msg="" package=0 cores=24 efficiency=16 threads=24
time=2026-01-22T17:57:55.258+08:00 level=WARN source=cuda_common.go:60 msg="old CUDA driver detected - please upgrade to a newer driver for best performance" version=12.8
time=2026-01-22T17:57:55.262+08:00 level=INFO source=types.go:131 msg="inference compute" id=GPU-d80e43d2-142e-01b2-36b5-b3c2b3234d75 library=cuda variant=v12 compute=12.0 driver=12.8 name="NVIDIA GeForce RTX 5080 Laptop GPU" total="15.9 GiB" available="14.6 GiB"
time=2026-01-22T17:57:55.262+08:00 level=INFO source=routes.go:1569 msg="entering low vram mode" "total vram"="15.9 GiB" threshold="20.0 GiB"
llama_model_loader: loaded meta data with 24 key-value pairs and 112 tensors from D:\\JZDSLx\\llm_models\\blobs\\sha256-970aa74c0a90ef7482477cf803618e776e173c007bf957f635f1015bfcfef0e6 (version GGUF V3 (latest))
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0:                          general.architecture str           = nomic-bert
llama_model_loader: - kv 1:                          general.name str              = nomic-embed-text-v1.5
llama_model_loader: - kv 2:                          nomic-bert.block_count u32                = 12
llama_model_loader: - kv 3:                          nomic-bert.context_length u32             = 2048
llama_model_loader: - kv 4:                          nomic-bert.embedding_length u32          = 768
llama_model_loader: - kv 5:                          nomic-bert.feed_forward_length u32        = 3072
```

RAG系统知识库构建+知识问答+智能出题+历史记录处理+问题知识图谱呈现

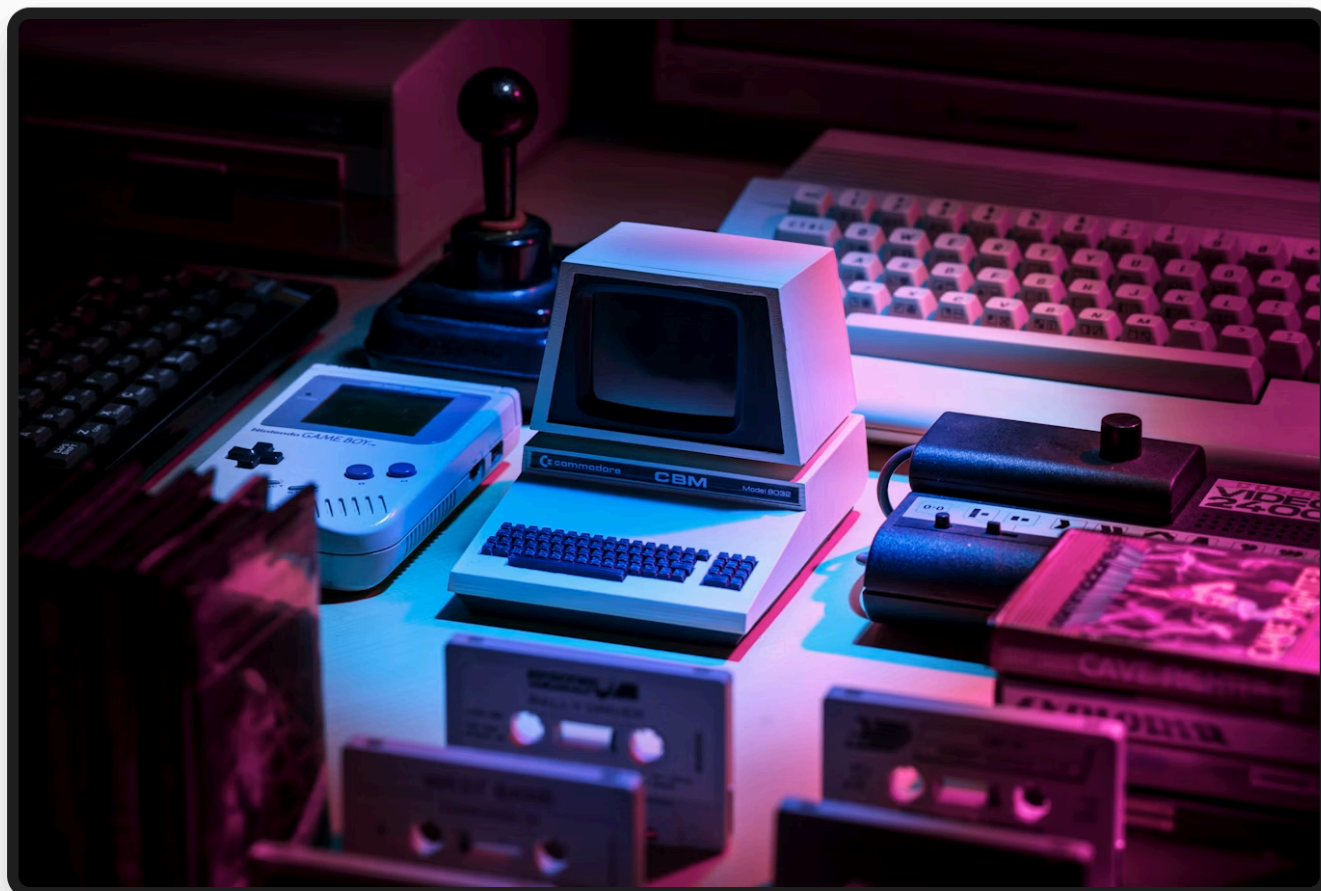
主界面简约大气可以通向 知识问答 和 知识图谱 两个界面

像乐高一样 构建知识图谱

好的设计，一定是有灵魂的。通过 RAG 技术与 Neo4j 图数据库，我们将文档转化为可视化的知识网络，让错题分析与知识检索变得直观、高效。

开始使用

查看图谱



知识问答界面左侧设置历史记录存储以及重要信息压缩
右侧显示对话输入界面

+ 新对话

历史记录

📄 N皇后问题

📄 动态规划相关题目讲解

U 我的账户

🗜️ RAG 算法导师 v2.1

🔄

🗜️

你好！我是你的算法导师。你可以问我具体的算法问题，也可以让我为你出题（输入"出题"）。

输入您的问题...

发送 ➤

对于最长公共子序列问题我们可以看到索引创建能够实现高效查找：并且索引到可视化讲解界面【1】

←

localhost:5175

🏠 首页

💬 知识问答

🗺️ 知识图谱

← RAG KNOWLEDGE

+ 新对话

历史记录

🗉 N皇后问题

🗉 动态规划相关题目讲解

RAG 算法导师 v2.1

lcs

👤

📖

最长公共子序列 (LCS) 详解与算法实现

Difficulty: Medium Tags: Dynamic Programming, String

1. LCS 定义

最长公共子序列，英文缩写为 LCS（Longest Common Subsequence）。其定义是，一个序列 S，如果分别是两个或多个已知序列的子序列，且是所有符合此条件序列中最长的，则 S 称为已知序列的最长公共子序列。

j	0	1	2	3	4	5	6
i	y _j	B	D	C	A	B	A
0	x _i	0	0	0	0	0	0
1	A	0	↑	↑	↑	←1	1
2	B	0	↖	←1	←1	↑	←2
3	C	0	↑	↑	2	←2	2

📊 算法可视化

GitHub

中文

EN

首页 · 字符串算法 · 最长公共子序列

最长公共子序列可视化

动态规划经典问题 - LCS

最长公共子序列 (LCS) 问题是寻找两个序列共有的最长子序列。子序列不要求连续，但必须保持相对顺序。LCS是许多文本比较和生物信息学应用的基础。

序列 A: ABCBDAB

序列 B: BDCABA

A: A B C B D A B

B: B D C A B A

DP 矩阵

		B	D	C	A	B	A
	0	0	0	0	0	0	0
A	0						
B	0						
C	0						
B	0						
D	0						

序列
A

ABCBDAB

序列
B

BDCABA

A:

A

B

C

B

D

A

B

B:

B

D

C

A

B

A

DP 矩阵

		B	D	C	A	B	A
		0	0	0	0	0	0
A		0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
B		0	↖ 1	← 1	← 1	↑ 1	↖ 2
C		0	↑ 1	↑ 1	↖ 2	← 2	↑ 2
B		0	↖ 1	↑ 1	↑ 2	↖ 3	← 3
D		0	↑ 1	↖ 2	↑ 2	↑ 3	↑ 3
A		0	↑ 1	↑ 2	↑ 2	↖ 3	↖ 4
B		0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4

最长公共子序列

B

D

A

B

长度: 长度: 4

代码出题部分可以根据难易程度进行筛选和随机生成：

localhost:5175

←

RAG KNOWLEDGE

🏠 首页

💬 知识问答

🌐 知识图谱

请按照难度梯度为：难中易，三档帮我出题

🗑️

好的，根据您的要求和提供的已知信息，我将分别为难题、中等题和简单题进行设计。每道题都会包含相应的图片链接以帮助理解题目背景。

📌

难题: Sliding Puzzle

难度: 难

描述: 给定一个 3x3 的滑动拼图，其中有一个空位（用 '0' 表示），其他位置填充了数字 1 到 8。你的目标是通过移动相邻的非零数字来将空位移到左上角的位置（即从 (2,2) 移到 (0,0)）。每次移动只能向空位的方向移动一个单位距离。

输入:

- 拼图状态，用一个包含 9 个整数的列表表示，例如: [1, 2, 3, 4, 5, 6, 7, 8, 0]

输出:

- 如果可以空位移到左上角，则返回 True；否则返回 False。

示例:

输入: [1, 2, 3, 4, 5, 6, 7, 8, 0]

输出: True

解释: 可以通过移动来实现目标。

参考图片:

1	2	3
4		5

请输入问题...

发送

←

RAG KNOWLEDGE

🏠 首页

💬 知识问答

🌐 知识图谱

1	2	3
4		5

📌

中等题: Deepest Leaves Sum

难度: 中

描述: 给定一棵二叉树，计算其最底层所有节点的和。

输入:

- 一个非空二叉树的根节点 root

输出:

- 最底层所有节点值之和

示例:

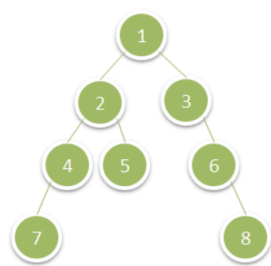
输入: [1, 2, 3, 4, 5, null, 6, 7, null, null, null, null, 8]

```
1
 / \
2   3
 / \
4   5
 \   \
 6   7
 /
8
```

请输入问题...

发送

8
/ 7
输出: 15 (4 + 5 + 6 + 7 = 22)
参考图片:



简单题: Out of Boundary Paths

难度: 易

描述: 一个机器人位于一个 $m \times n$ 的网格的左上角 $(0, 0)$ 。每次移动可以向上、下、左或右移动一格, 但不能离开边界。计算从起点到终点的所有可能路径数。

输入:

- 网格的行数 m
- 网格的列数 n

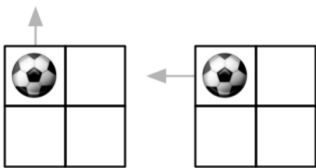
输出:

- 返回所有可能路径的数量

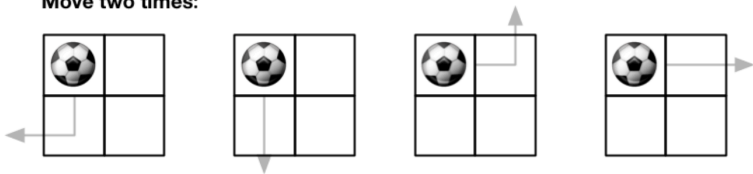
请输入问题...

发送

Move one time:



Move two times:



希望这些题目符合您的要求! 如果有任何其他需求或修改, 请随时告知。



问题分类相关的知识图谱呈现:

知识图谱概览 (42 节点 / 39 关系)

 刷新

